

GenomicRanges - GRanges

Kasper D. Hansen

- [Dependencies](#)
- [Corrections](#)
- [Other Resources](#)
- [GRanges](#)
- [GRanges, seqinfo](#)
- [SessionInfo](#)
- [References](#)

Dependencies

This document has the following dependencies:

```
library(GenomicRanges)
```

Use the following commands to install these packages in R.

```
source("http://www.bioconductor.org/biocLite.R")
biocLite(c("GenomicRanges"))
```

Corrections

Improvements and corrections to this document can be submitted on its [GitHub](#) in its [repository](#).

Other Resources

- The vignettes from the [GenomicRanges webpage](#).
- The package is described in a paper in PLOS Computational Biology (Lawrence et al. 2013).

GRanges

GRanges are like IRanges with strand and chromosome. Strand can be +, - and *. The value * indicates 'unknown strand' or 'unstranded'. This value usually gets treated as a third strand, which is sometimes confusing to users (examples below).

They get created with the GRanges constructor:

```
gr <- GRanges(seqnames = "chr1", strand = c("+", "-", "+"),
              ranges = IRanges(start = c(1,3,5), width = 3))
```

Natural accessor functions: strand(), seqnames(), ranges(), start(), end(), width().

Because they have strand, we now have operations which are relative to the direction of transcription (upstream(), downstream()):

```
flank(gr, 2, start = FALSE)
```

```
## GRanges object with 3 ranges and 0 metadata columns:
##      seqnames      ranges strand
##      <Rle> <IRanges> <Rle>
## [1]   chr1     [4, 5]      +
## [2]   chr1     [1, 2]      -
## [3]   chr1     [8, 9]      +
## -----
## seqinfo: 1 sequence from an unspecified genome; no seqlengths
```

GRanges, seqinfo

GRanges operate within a universe of sequences (chromosomes/contigs) and their lengths.

This is described through seqinfo:

```
seqinfo(gr)
```

```
## Seqinfo object with 1 sequence from an unspecified genome; no seqlengths
##   seqnames seqlengths isCircular genome
##   chr1      NA        NA      <NA>
```



```
seqlengths(gr) <- c("chr1" = 10)
seqinfo(gr)
```

```
## Seqinfo object with 1 sequence from an unspecified genome:
##   seqnames seqlengths isCircular genome
##   chr1      10        NA      <NA>
```

```
seqlevels(gr)
```

```
## [1] "chr1"
```

```
seqlengths(gr)
```

```
## chr1
##    10
```

Especially the length of the chromosomes are used by some functions. For example `gaps()` return the stretches of the genome not covered by the `GRanges`.

```
gaps(gr)
```

```
## GRanges object with 5 ranges and 0 metadata columns:
##      seqnames      ranges strand
##      <Rle> <IRanges> <Rle>
## [1]   chr1      [4, 4]      +
## [2]   chr1      [8, 10]     +
## [3]   chr1      [1, 2]      -
## [4]   chr1      [6, 10]     -
## [5]   chr1      [1, 10]     *
## -----
## seqinfo: 1 sequence from an unspecified genome
```

In this example, we know that the last gap stops at 10, because that is the length of the chromosome. Note how a range on the * strand appears in the result.

Let us expand the GRanges with another chromosome

```
seqlevels(gr) <- c("chr1", "chr2")
seqnames(gr) <- c("chr1", "chr2", "chr1")
```

When you `sort()` a GRanges, the sorting order of the chromosomes is determined by their order in `seqlevel`. This is nice if you want the sorting “chr1”, “chr2”, ..., “chr10”, ...

```
sort(gr)
```

```
## GRanges object with 3 ranges and 0 metadata columns:
##      seqnames      ranges strand
##      <Rle> <IRanges> <Rle>
## [1]   chr1      [1, 3]      +
## [2]   chr1      [5, 7]      +
## [3]   chr2      [3, 5]      -
## -----
## seqinfo: 2 sequences from an unspecified genome
```

```
seqlevels(gr) <- c("chr2", "chr1")
sort(gr)
```

```
## GRanges object with 3 ranges and 0 metadata columns:
##      seqnames      ranges strand
##      <Rle> <IRanges> <Rle>
## [1]   chr2      [3, 5]      -
## [2]   chr1      [1, 3]      +
## [3]   chr1      [5, 7]      +
## -----
## seqinfo: 2 sequences from an unspecified genome
```

You can associate a genome with a GRanges.

```
genome(gr) <- "hg19"
gr
```

```
## GRanges object with 3 ranges and 0 metadata columns:
##      seqnames      ranges strand
##      <Rle> <IRanges> <Rle>
## [1]   chr1      [1, 3]      +
## [2]   chr2      [3, 5]      -
## [3]   chr1      [5, 7]      +
## -----
## seqinfo: 2 sequences from hg19 genome
```

This becomes valuable when you deal with data from different genome versions (as we all do), because it allows R to throw an error when you compare two GRanges from different genomes, like

```
gr2 <- gr
genome(gr2) <- "hg18"
findOverlaps(gr, gr2)
```

```
## Error in mergeNamedAtomicVectors(genome(x), genome(y), what = c("sequ
## - in 'x': hg19, hg19
## - in 'y': hg18, hg18
```

The fact that each sequence may have its own genome is more esoteric. One usecase is for experiments where the experimenter have spiked in sequences exogenous to the original organism.

SessionInfo

```
## R version 3.2.1 (2015-06-18)
## Platform: x86_64-apple-darwin13.4.0 (64-bit)
## Running under: OS X 10.10.5 (Yosemite)
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## attached base packages:
## [1] stats4      parallel  methods    stats      graphics  grDevices  utils
## [8] datasets   base
##
## other attached packages:
## [1] GenomicRanges_1.20.5 GenomeInfoDb_1.4.2  IRanges_2.2.7
## [4] S4Vectors_0.6.3      BiocGenerics_0.14.0 BiocStyle_1.6.0
## [7] rmarkdown_0.7
##
## loaded via a namespace (and not attached):
## [1] digest_0.6.8      formatR_1.2        magrittr_1.5       evaluate_0.7.2
## [5] stringi_0.5-5     xvector_0.8.0      tools_3.2.1        stringr_1.0.0
## [9] yaml_2.1.13       htmltools_0.2.6    knitr_1.11
```



References

Lawrence, Michael, Wolfgang Huber, Hervé Pagès, Patrick Aboyoun, Marc Carlson, Robert Gentleman, Martin T Morgan, and Vincent J Carey. 2013. "Software for computing and annotating genomic ranges." *PLoS Computational Biology* 9 (8): e1003118. doi:10.1371/journal.pcbi.1003118.